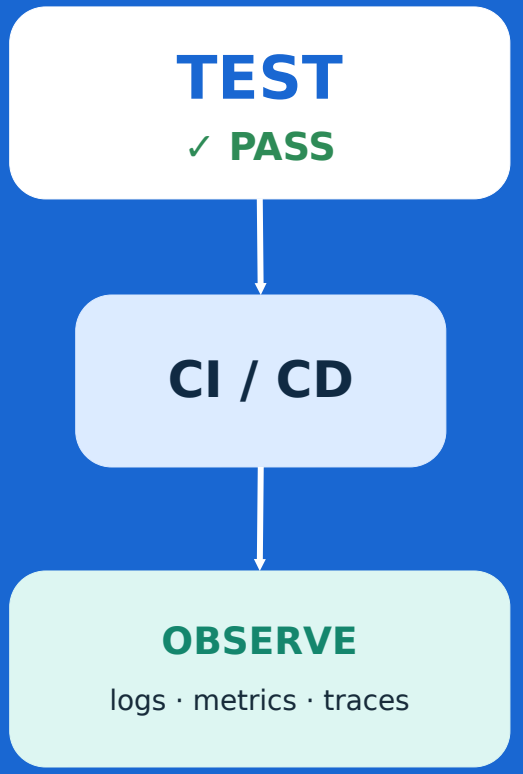
Тестування програмних продуктів

Сучасні методи, автоматизація, CI/CD, DevSecOps та AI

Навчальна презентація



Структура теми

01 **Основи**

поняття, цілі, принципи

02 **Процес**

STLC, артефакти, дефекти

03 **Методи**

рівні, види, техніки

04 **Автоматизація**

UI, API, контракти, CI/CD

05 **Якість системи**

performance, security, accessibility

06 **Нові підходи**

cloud, observability, AI

Визначення тестування

Тестування програмного забезпечення

систематичний процес аналізу й оцінювання програмного продукту, спрямований на виявлення дефектів, перевірку вимог і зменшення ризиків його використання.



Тестування надає докази для рішення про реліз, але не доводить повної відсутності помилок.

QA, QC і тестування

Поняття	Основний фокус	Типові дії	Результат
Quality Assurance	Процеси	стандарти, аудити, профілактика	менше причин дефектів
Quality Control	Готовий продукт	інспекції, вимірювання, перевірки	оцінка відповідності
Software Testing	Поведінка системи	виконання тестів, аналіз результатів	інформація про якість і ризик

QA формує процеси. QC оцінює результат. Тестування створює докази про поведінку продукту.

Верифікація та валідація

ВЕРИФІКАЦІЯ

«Чи правильно ми створюємо продукт?»

- рев’ю вимог і дизайну
- статичний аналіз коду
- перевірка за специфікацією

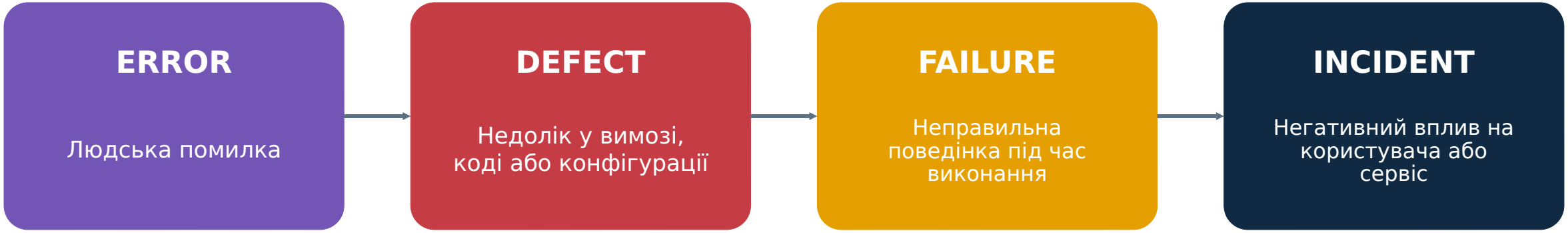
ВАЛІДАЦІЯ

«Чи створили ми потрібний продукт?»

- приймальне тестування
- перевірка бізнес-сценаріїв
- оцінювання користувачами

Приклад: формула відповідає специфікації, але сама специфікація не розв’язує задачу користувача.

Від людської помилки до інциденту



Root Cause Analysis шукає процесну або технічну першопричину, а не лише зовнішній симптом.

Сім принципів тестування

01 **Тести виявляють дефекти, але не доводять їх відсутність**

02 Повне тестування практично неможливе

03 Раннє тестування зменшує витрати

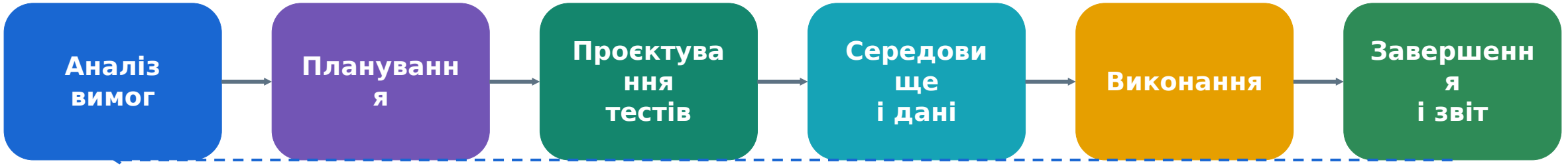
04 Дефекти концентруються у складних модулях

05 Набори тестів потрібно оновлювати

06 Методи залежать від контексту

07 Бездефектний продукт може не мати цінності

Життєвий цикл тестування STLC



Зворотний зв’язок повертає команду до вимог і плану після кожної суттєвої знахідки.

Планування та артефакти тестування

Артефакт	Що містить	Для чого використовується
Test Strategy	принципи, рівні, інструменти	єдині правила для продукту
Test Plan	обсяг, ресурси, терміни, ризики	керування конкретним циклом
Checklist	короткий перелік перевірок	швидка перевірка та дослідження
Test Case	умови, кроки, очікуваний результат	відтворювана детальна перевірка
Traceability Matrix	зв’язок вимог і тестів	контроль покриття
Test Report	результати, дефекти, залишкові ризики	рішення про реліз

Структура якісного тест-кейсу

- **ID і назва**
- **Передумови**
- **Тестові дані**
- **Кроки виконання**
- **Очікуваний результат**
- **Фактичний результат і статус**

Приклад

Назва: блокування після п’яти невдалих входів

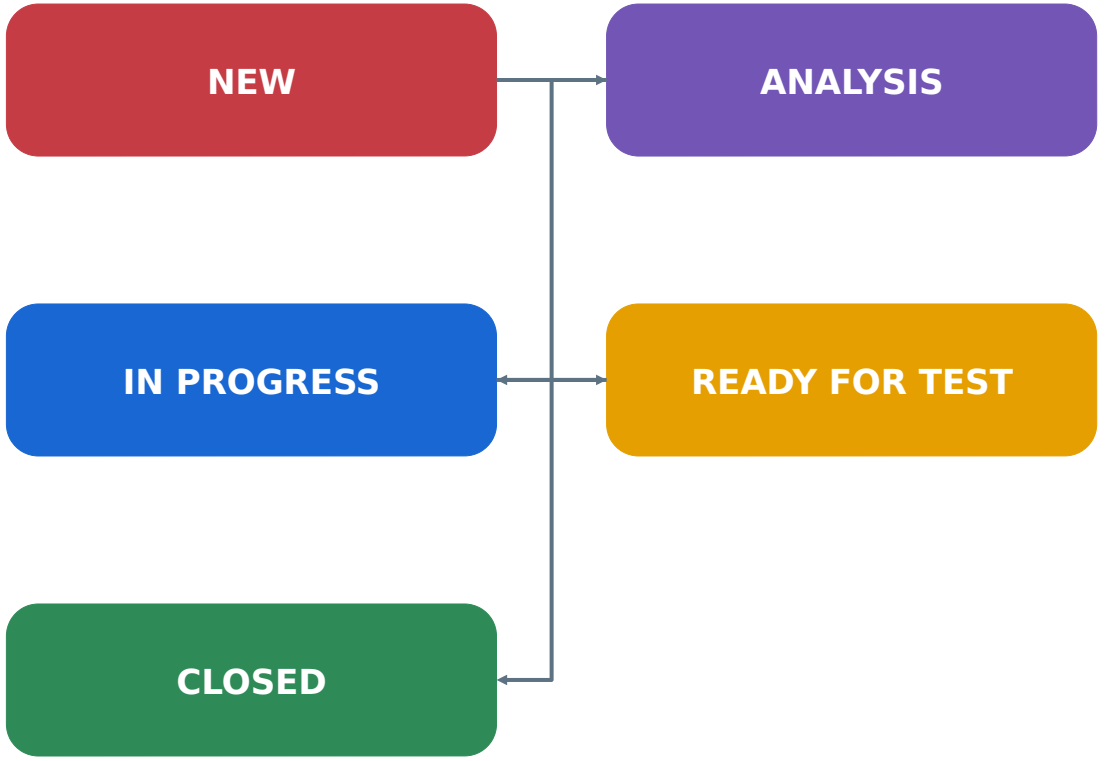
Передумова: активний обліковий запис

Кроки: п’ять разів ввести неправильний пароль

Очікування: система блокує вхід на 10 хвилин і реєструє подію безпеки

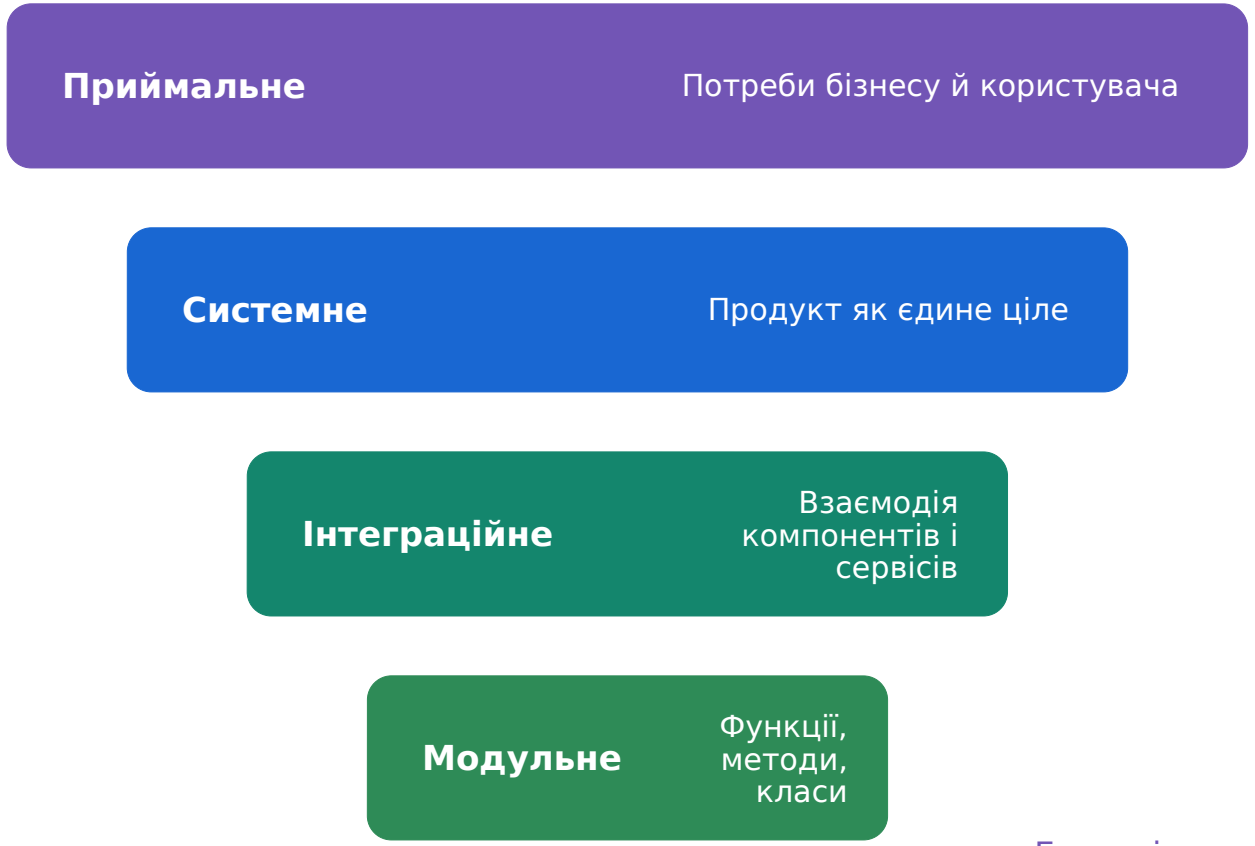
Звіт і життєвий цикл дефекту

- коротка й точна назва
- версія продукту та середовище
- кроки відтворення
- очікуваний і фактичний результат
- severity, priority та докази



Severity описує вплив. Priority визначає терміновість виправлення.

Рівні тестування



Швидкість виконання зростає вниз

Близькість до реальної поведінки зростає вгору

Функціональне та нефункціональне тестування

Критерій	Функціональне	Нефункціональне
Головне питання	Що робить система?	Наскільки якісно вона це робить?
Основа	бізнес-вимоги, use cases	атрибути якості, SLO, стандарти
Приклади	вхід, пошук, платіж, звіт	швидкодія, безпека, доступність
Типові результати	правильність обчислень і станів	latency, throughput, ризики, usability

Один бізнес-сценарій потребує обох груп перевірок: платіж має бути правильним, швидким, безпечним і доступним.

Статичне та динамічне тестування

Ознака	Статичне	Динамічне
Запуск програми	не потрібний	обов’язковий
Об’єкти	вимоги, дизайн, код, конфігурації	запущені компоненти та система
Методи	review, inspection, lint, SAST	unit, API, UI, load, DAST
Перевага	раннє виявлення причин дефектів	перевірка реальної поведінки
Обмеження	не показує runtime-проблеми	вимагає середовища та даних

Найкраще покриття дає поєднання статичних і динамічних методів у спільному pipeline.

Black box, white box i gray box

BLACK BOX

Внутрішня реалізація невідома

вимоги, входи, виходи

WHITE BOX

Структура коду відома

гілки, умови, потоки даних

GRAY BOX

Часткове знання архітектури

API, БД, інтеграції

Вибір підходу визначають мета перевірки, доступна інформація та роль тестувальника.

Техніки проєктування тестів

Класи еквівалентності

скорочують кількість подібних значень

Граничні значення

перевіряють точки зміни поведінки

Таблиці рішень

охоплюють комбінації умов

Переходи станів

перевіряють дозволені й заборонені переходи

Pairwise

покриває всі пари параметрів

Property-based testing

перевіряє загальні властивості на багатьох даних

Mutation testing

оцінює здатність тестів помічати зміни коду

Класи еквівалентності та межі

Умова: допустимий вік користувача від 18 до 65 років



Недопустимий клас **Допустимий клас** **Недопустимий клас**

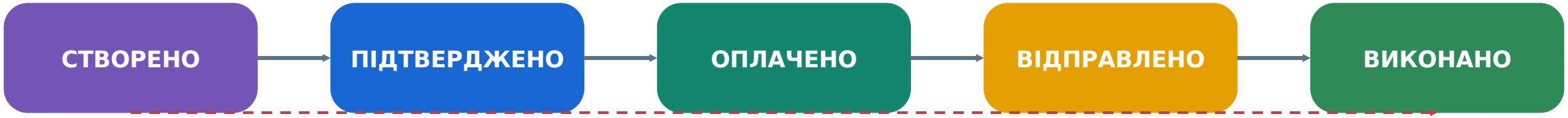
Мінімальний набір навколо меж: 17, 18, 19, 64, 65, 66

Таблиця рішень: приклад платежу

Авторизація	Коштів достатньо	Ліміт не перевищено	Очікуваний результат
Так	Так	Так	Платіж дозволено
Так	Ні	Так	Відмова: недостатньо коштів
Так	Так	Ні	Відмова: перевищено ліміт
Ні	—	—	Відмова: потрібна авторизація

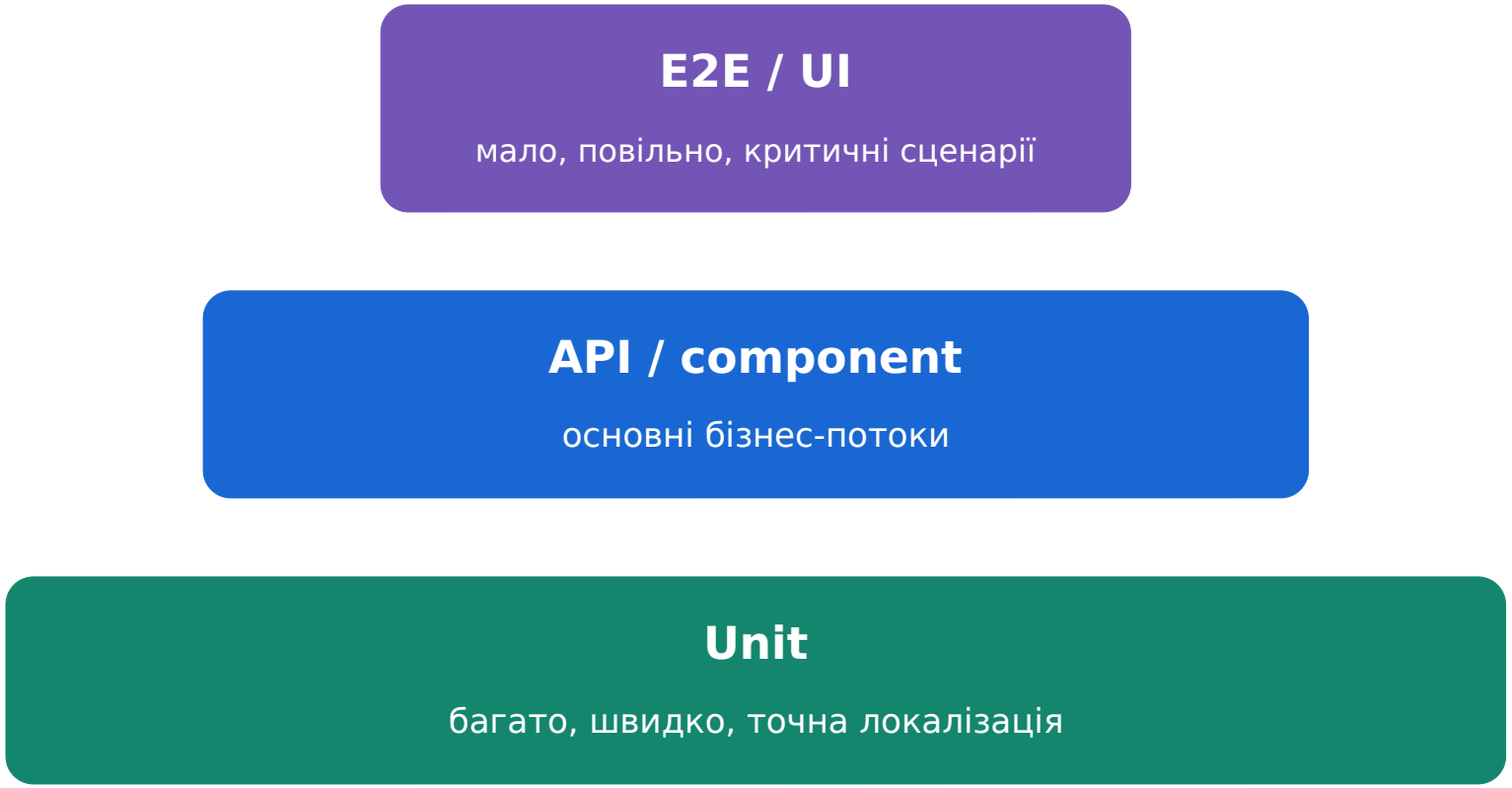
Таблиця робить бізнес-правила явними та допомагає знайти пропущені комбінації умов.

Тестування переходів станів



Заборонений перехід: СТВОРЕНО → ВИКОНАНО

Стратегія автоматизації



Ручне дослідницьке тестування

- нові ризики
- складні взаємодії
- суб’єктивна якість

Автоматизують повторювані перевірки, які дають швидкий і стабільний зворотний зв’язок.

Тестування вебінтерфейсів

Playwright

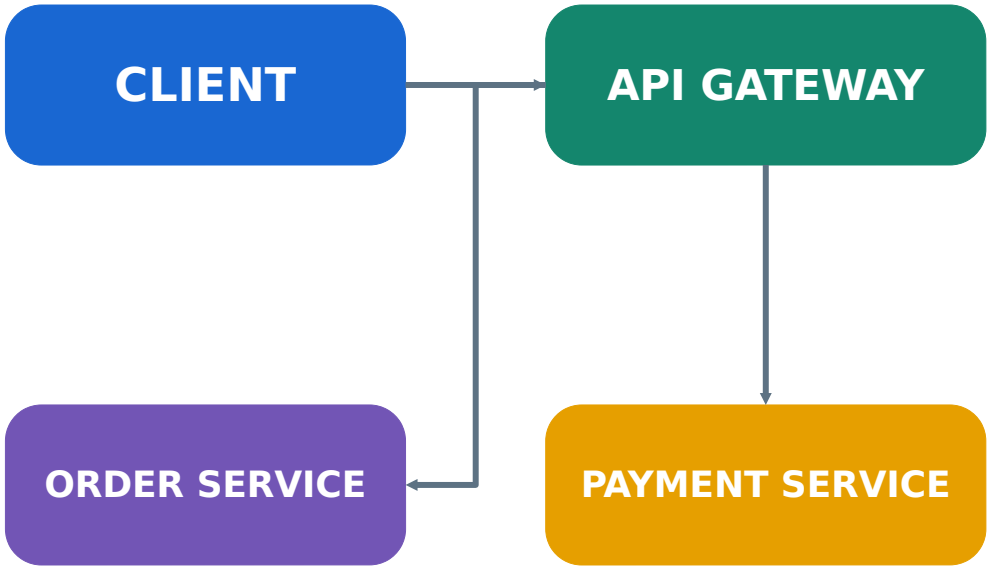
- Chromium, Firefox і WebKit
- auto-waiting та web-first assertions
- ізольовані browser contexts
- Trace Viewer для аналізу CI-помилок
- семантичні локатори role, label, test id

```
test('вхід до системи', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill(user);
  await page.getByLabel('Пароль').fill(pass);
  await page.getByRole('button', { name:
'Увійти' }).click();
  await
expect(page.getByText('Кабінет')).toBeVisible();
});
```

Стабільний тест описує поведінку користувача, а не випадкову структуру DOM.

API та мікросервіси

- HTTP-метод і статус-код
- структура та схема відповіді
- валідація і помилки
- автентифікація та авторизація
- ідемпотентність
- пагінація і rate limit
- версійна сумісність



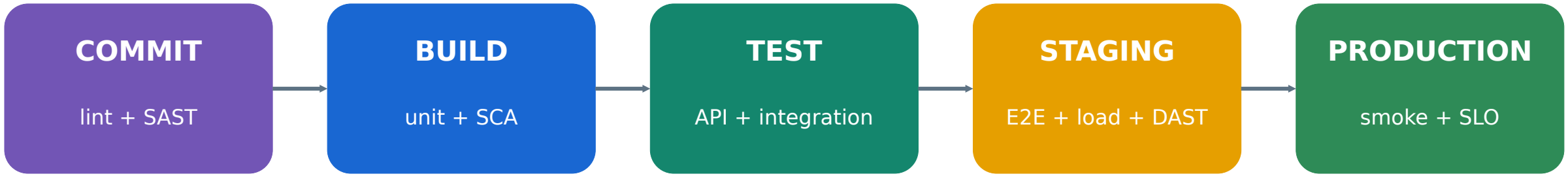
API-тести швидші та точніше локалізують помилки, ніж повні UI-сценарії.

Контрактне тестування мікросервісів



Pact перевіряє сумісність сервісів без запуску всієї розподіленої системи.

Безперервне тестування в CI/CD



Quality gate зупиняє зміни, якщо порушені критерії якості, безпеки або продуктивності.

Тестування продуктивності

Вид	Мета	Приклад
Load	робота під очікуваним навантаженням	2 000 активних користувачів
Stress	пошук межі працездатності	зростання до відмови
Spike	реакція на різкий стрибок	десятикратний пік трафіку
Soak	стабільність у часі	навантаження протягом 24 годин
Scalability	ефект додавання ресурсів	подвоєння кількості pod

Ключові метрики

- **Latency p95 / p99**
- **Throughput, RPS**
- **Error rate**
- **CPU, RAM, DB pool**
- **Recovery time**

p95 = 800 мс означає: 95% запитів завершилися не довше ніж за 800 мс.

Тестування безпеки та DevSecOps

Метод	Що аналізує	Приклади інструментів
SAST	вихідний код без запуску	Semgrep, CodeQL, SonarQube
DAST	запущений застосунок зовнішніми запитами	OWASP ZAP, Burp Suite
SCA	залежності, CVE та ліцензії	Trivy, Snyk, Dependency-Check
Secret scanning	ключі, токени, паролі у репозиторії	Gitleaks, GitHub secret scanning
SBOM	склад і походження компонентів	CycloneDX, SPDX, Syft

Security checks входять до pipeline, а критичні знахідки блокують збірку або реліз.

Тестування доступності: WCAG 2.2

PERCEIVABLE — **Сприйнятність**

альтернативний текст, контраст, субтитри

OPERABLE — **Керованість**

клавіатура, фокус, розмір цілі

UNDERSTANDABLE — **Зрозумілість**

підказки, помилки, передбачуваність

ROBUST — **Надійність**

семантика, сумісність assistive tech

Автоматизовані перевірки доповнюють ручну роботу з клавіатурою та screen reader.

Cloud-native testing і chaos engineering

APPLICATION — API, бізнес-логіка, data consistency

PLATFORM — Kubernetes, probes, autoscaling, service mesh

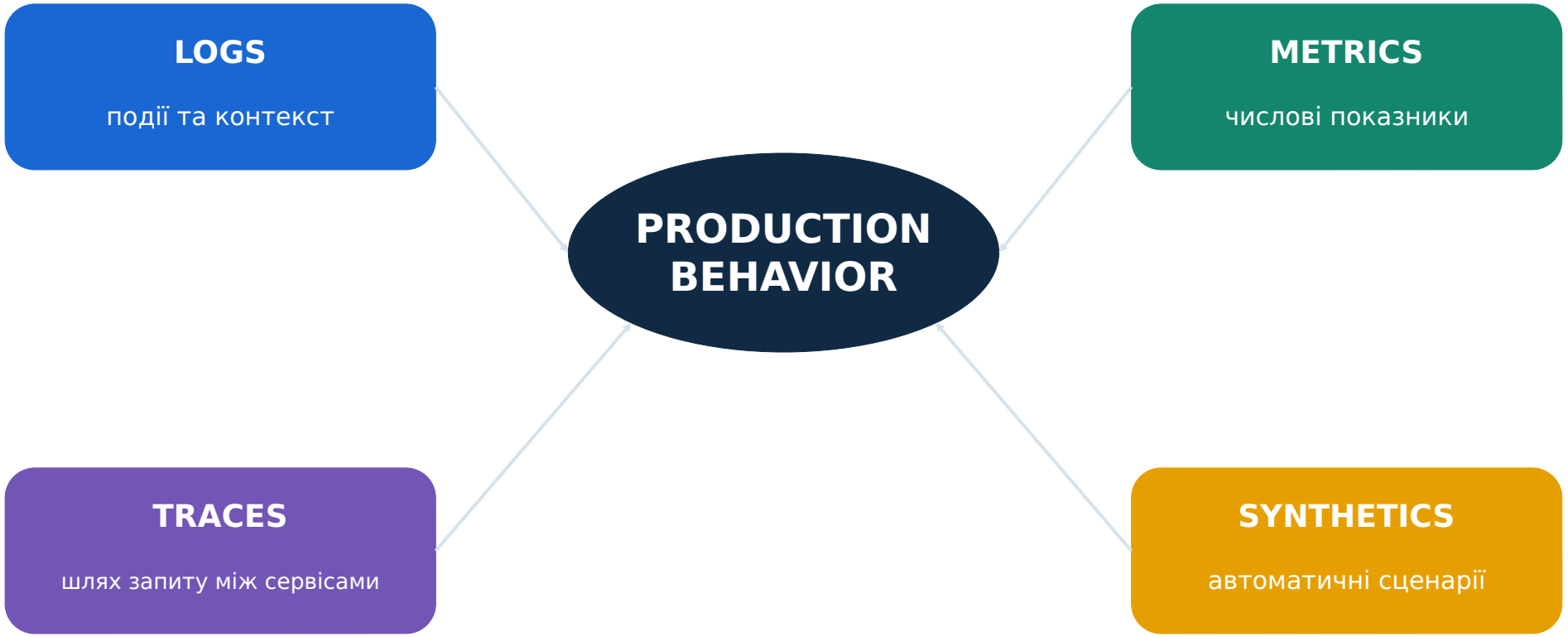
INFRA — мережа, storage, CPU, пам’ять

Chaos experiment



Приклади: втрата мережі, падіння pod, затримка БД, дефіцит пам’яті.

Shift Right та спостережуваність



Canary release + SLO + автоматичний rollback зменшують ризик змін.

Штучний інтелект у роботі тестувальника

- генерація тестових ідей із вимог
- створення граничних даних
- пояснення падіння тестів
- пошук дублікатів дефектів
- пріоритизація регресії
- аналіз логів і візуальних змін

AI-пропозиції потребують перевірки: модель може вигадати вимогу, пропустити ризик або створити слабкий oracle.

Тестування AI та генеративних систем

Напрям	Що оцінюють
Якість моделі	precision, recall, F1, calibration
Дані	репрезентативність, витоки, drift
Справедливість	bias і різницю результатів між групами
Стійкість	шум, adversarial input, нестандартні сценарії
GenAI	фактичність, релевантність, токсичність, формат
Безпека LLM	prompt injection, data leakage, tool abuse
Експлуатація	latency, вартість, model drift, rollback

Методи оцінювання

- еталонні набори
- автоматичні метрики
- експертна оцінка
- red teaming
- production monitoring

Імовірнісна система потребує порогів, вибірки та повторюваних evals.

Тестові дані та конфіденційність

Покриття даних

- нормальні й граничні значення
- порожні та неправильні типи
- Unicode і спеціальні символи
- дублікати й конфліктні стани
- великі обсяги й різні часові пояси

Захист даних

- маскування
- псевдонімізація
- синтетична генерація
- мінімізація підмножин
- контроль доступу й термінів зберігання

Копія production-бази без захисту персональних даних створює технічний і правовий ризик.

Метрики та ризик-орієнтоване тестування

Покриття вимог

C_r = перевірені вимоги / усі вимоги × 100%

Defect Removal Efficiency

DRE = дефекти до релізу / усі знайдені дефекти × 100%

Ризик

$$R_i = P_i \times I_i$$



Сучасний набір інструментів

Напрям	Інструменти	Напрям	Інструменти
Unit	JUnit, pytest, Vitest	Web UI	Playwright, Cypress, Selenium
API	Postman, REST Assured, Karate	Mobile	Appium, Espresso, XCUITest
Contracts	Pact, Spring Cloud Contract	Performance	k6, JMeter, Gatling, Locust
Security	ZAP, Semgrep, CodeQL, Trivy	Accessibility	axe-core, Lighthouse, Pa11y
Mutation	PIT, Stryker, mutmut	Containers	Testcontainers
Chaos	Chaos Mesh, LitmusChaos	Observability	OpenTelemetry, Prometheus, Grafana
CI/CD	GitHub Actions, GitLab CI, Jenkins	Reports	Allure, ReportPortal

Інструмент обирають після визначення ризику, рівня тесту, технологічного стека та вартості підтримки.

Критерії готовності до релізу

- ✓ **виконані критерії приймання**
- ✓ **покриті критичні вимоги**
- ✓ **немає відкритих blocker і critical дефектів**
- ✓ **регресійний набір пройдено**
- ✓ **security-ризики прийнятні**
- ✓ **performance відповідає SLO**
- ✓ **перевірені backup і recovery**
- ✓ **готові monitoring та rollback**

Рішення про реліз враховує залишковий ризик, а не лише кількість успішних тестів.

Висновки

Сучасне тестування є безперервною системою зворотного зв’язку

РАНО

аналіз вимог, review, static analysis

ЧАСТО

автоматизовані перевірки в CI/CD

ГЛИБОКО

API, security, performance, accessibility

ПІСЛЯ РЕЛІЗУ

SLO, observability, canary і rollback

Мета процесу: швидко отримувати достовірну інформацію про якість і безпечно випускати зміни.

Основні джерела: ISO/IEC/IEEE 29119-1:2022 · WCAG 2.2 · NIST SSDF · OWASP WSTG · OpenTelemetry